

PART A

(PART A : TO BE REFERRED BY STUDENTS)

Experiment No. 1(A)

Aim: To design and implement first pass of a two pass assembler for IBM 360/370 Processor

Objective: Develop a program to implement first pass:

- a. To search instruction in MOT and return its length
- b. To search instruction in POT and return the routine called
- c. To generate Symbol table
- d. To generate literal table
- e. To generate intermediate code after pass 1

Outcome: Students are able to design and implement pass 1 of two pass assembler.

Theory:

An assembler performs the following functions

1. Generate instructions
 - a. Evaluate the mnemonic in the operator field to produce its machine code.
 - b. Evaluate subfields- find value of each symbol, process literals & assign address.
2. Process pseudo ops.

Pass 1: Purpose - To define symbols & literals

1. Determine length of machine instruction (MOTGET)
2. Keep track of location counter (LC)
3. Remember values of symbols until pass2 (STSTO)
4. Process some pseudo ops. EQU
5. Remember literals (LITSTO)

Pass 1: Database

1. Source program
2. Location counter(LC) which stores location of each instruction
3. Machine Operation Table (MOT). This table indicates the symbolic mnemonic for each instructions and its length.
4. Pseudo Operation Table (POT). This table indicates the symbolic mnemonic and action taken for each pseudo-op in pass1.
5. Symbol Table (ST) which stores each label along with its value.
6. Literal Table(LT) which stores each literal and its corresponding address
7. A copy of input which will be used by pass2.

Format of databases

The Machine Operation Table (MOT) and Pseudo Operation Table (POT) are examples of fixed tables. During the assembly process the contents of this table are not filled in or altered

1. **Machine-op Table (MOT)**

6-bytes per entry				
Mnemonic -codes (4 bytes) characters	Binary op-codes (1 byte) hexadecimal	Instruction length (2-bits) binary	Instruction format (3-bits) binary	Not used in this design (3 bits)
"Abbb"	5A	10	001	
"AHbb"	4A	10	001	
"ALbb"	5E	10	001	
"ALRb"	1E	01	000	
.....				

b : bank space

2. Pseudo-op Table (POT)

8-BYTES PER ENTRY

Pseudo-op (5-bytes) Characters	Address of routine to process pseudo-op (3 bytes= 24 bits address)
"DROPb"	P1DROP
"ENDbb"	P1END
"EQUbb"	P1EQU
"START"	P1START
"USING"	P1USING

Let us consider following source code and find the contents of symbol table and literal table.

Stmt no	Symbol	Op-code	Operands
1	SAMPLE	START	0
2		USING	*,15
3		A	1,FOUR
4		A	2,FIVE
5	TEMP	EQU	10
6		A	3,=F'3'
7		USING	TEMP,15
8	FOUR	DC	F'4'

9	FIVE	DC	F'5'
10		END	

3. Symbol Table:

Symbol	Value	Length	Relocation
"SAMPLEbbb"	0	1	"R"
"TEMPbbbb"	10	4	"A"
"FOURbbbb"	12	4	"R"
"FIVEbbbb"	16	4	"R"

4. Literal Table

Literal	Value	Length	Relocation
F'3'	20	4	"R"

5. Code after pass1:

Stmt no	Relative address	Statement
1		SAMPLE START 0
2		USING *,15
3	0	A 1, _ (0,15)
4	4	A 2, _ (0,15)
5		-
6	8	A 3, _ (0,15)
7		-
8	12	FOUR 4
9	16	FIVE 5
10		-

Algorithm:

- Initially location counter is set to relative address 0 i.e. LC=0
- Read the statement from source program
- Examine the op-code field: If match found in MOT then
 - From the MOT entry determine the length field i.e. L=length.
 - Examine the operand field to check whether literal is present or not. If any new

literal is found then corresponding entry is done in LT.

c. Examine the label field for the presence of symbol. If label is present then it is entered in ST and current value of location counter is assigned to symbol. d. The current value of location counter is incremented by length of instruction(L) 4. If match found in POT then

- a. If it is USING or DROP pseudo-op then first pass do nothing. It just writes a copy of these cards for pass 2.
- b. If it is EQU pseudo-op then evaluate expression in operand field and assign value to the symbol present in label field.
- c. If it is DS or DC pseudo-op then by examining the operand field find out number of bytes of storage required. Adjust the location counter for proper alignment. d. If it is END pseudo-op then pass1 is terminated and control is passed to pass2. Before transferring the control it assigns location to literals.

5. A copy of source card is saved for pass 2.

6. Go to step 2.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No : C-54	Name: Quazi Moinuddin
Class: TE Comps C	Batch: C-3
Date of Experiment:29.01.5	Date of Submission: 28.02.25
Grade:	

B.1 Software Code written by student:

C code:

```
#include <stdio.h>
#include <stdlib.h> // For atoi() and exit()
#include <string.h> // For strcmp() and strcpy()
```

```
struct symtab {
    char sname[10];
    int addr;
} stab[10];
```

```
struct littab {
    char lname[10];
```

```

    int addr;
} ltab[10];

int main(void)
{
    int lc = 0, i = 0, j = 0, x;
    FILE *fp1, *fp2;
    char str[80];
    char s1[20], s2[20], s3[20]; // Allocate memory for tokens
    int reg[16] = {0};

    // Removed clrscr() because it's non-standard

    fp1 = fopen("pg1.ASM", "r");
    if (fp1 == NULL) {
        perror("Error opening pg1.ASM");
        exit(1);
    }

    fp2 = fopen("new.txt", "w");
    if (fp2 == NULL) {
        perror("Error opening new.txt");
        fclose(fp1);
        exit(1);
    }

    while (fgets(str, sizeof(str), fp1) != NULL)
    {
        /* Clear token arrays before using them */
        s1[0] = s2[0] = s3[0] = '\0';

        /* Extract tokens from the current line */
        sscanf(str, "%s %s %s", s1, s2, s3);

        if (strcmp(s2, "START") == 0)
        {
            lc = atoi(s3);
            strcpy(stab[i].sname, s1);
            stab[i].addr = lc;
            i++;
        }
        else if (strcmp(s1, "USING") == 0)
        {

```

```

        reg[15] = 1;
    }
    else if (strcmp(s2, "DC") == 0 || strcmp(s2, "DS") == 0)
    {
        strcpy(stab[i].sname, s1);
        stab[i].addr = lc;
        i++;
        lc += 4;
    }
    else if (strcmp(s1, "END") == 0)
    {
        for (x = 0; x < j; x++)
        {
            ltab[x].addr = lc;
            lc += 4;
        }
    }
    else
    {
        fprintf(fp2, "%d %s", lc, str);
        if (s3[0] == '=')
        {
            strcpy(ltab[j].lname, s3);
            j++;
        }
        lc += 4;
    }
}

```

// Replace fcloseall() with explicit file closures

```

fclose(fp1);
fclose(fp2);

```

```

printf("\nSymbol Table\n");
for (x = 0; x < i; x++)
    printf("%s %d\n", stab[x].sname, stab[x].addr);

```

```

printf("\nLiteral Table\n");
for (x = 0; x < j; x++)
    printf("%s %d\n", ltab[x].lname, ltab[x].addr);

```

```

return 0;
}

```

New text file

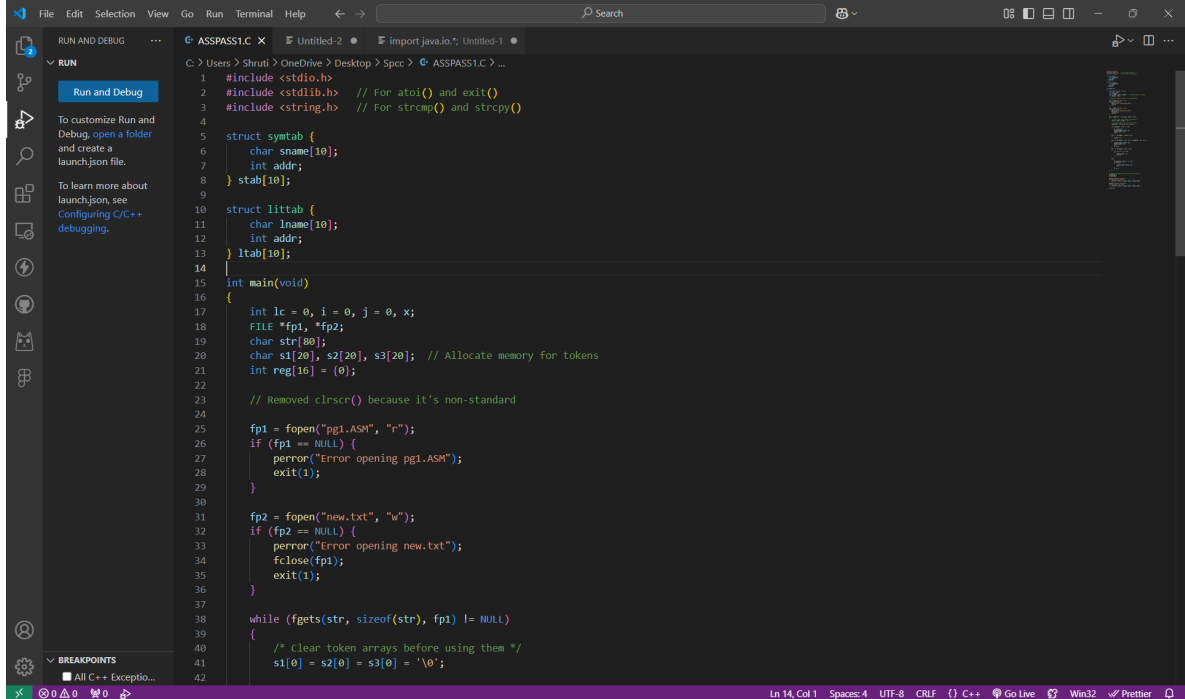
1000	L	1, FOUR
1004	A	1, ='5'
1008	ST	1, TEMP
1012	A	1, ='4'

PG1.ASM

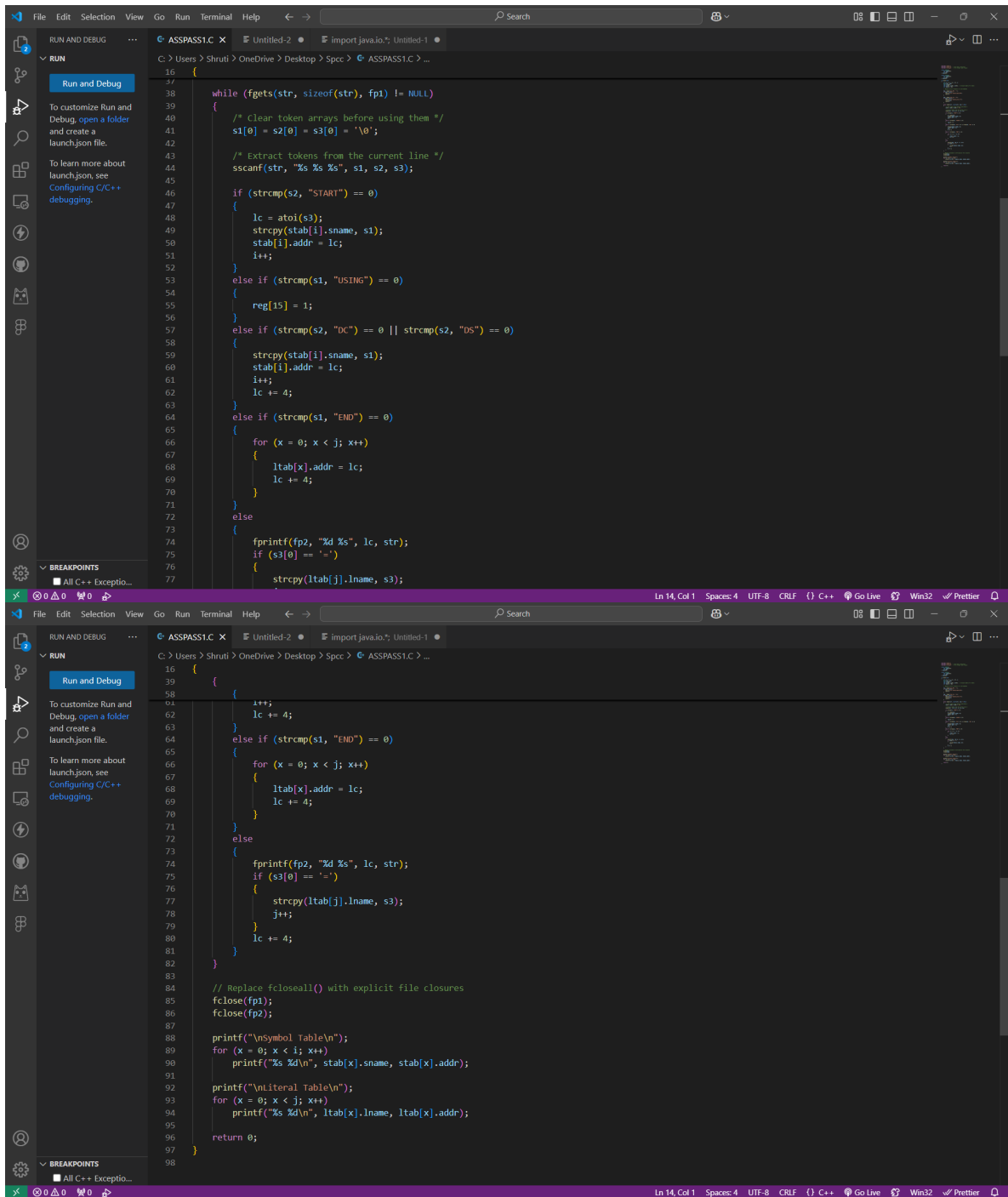
PG1	START	1000
	USING	*, 15
	L	1, FOUR
	A	1, ='5'
	ST	1, TEMP
	A	1, ='4'
FOUR	DC	F'4'
TEMP	DS	'1'F
	END	

B.2 Input and Output:

Input:



```
1  #include <stdio.h>
2  #include <stdlib.h> // For atoi() and exit()
3  #include <string.h> // For strcmp() and strcpy()
4
5  struct sytab {
6      char sname[10];
7      int addr;
8  } stab[10];
9
10 struct ltab {
11     char lname[10];
12     int addr;
13 } ltab[10];
14
15 int main(void)
16 {
17     int lc = 0, i = 0, j = 0, x;
18     FILE *fp1, *fp2;
19     char str[80];
20     char s1[20], s2[20], s3[20]; // Allocate memory for tokens
21     int reg[16] = {0};
22
23     // Removed clrscr() because it's non-standard
24
25     fp1 = fopen("pg1.ASM", "r");
26     if (fp1 == NULL) {
27         perror("Error opening pg1.ASM");
28         exit(1);
29     }
30
31     fp2 = fopen("new.txt", "w");
32     if (fp2 == NULL) {
33         perror("Error opening new.txt");
34         fclose(fp1);
35         exit(1);
36     }
37
38     while (fgets(str, sizeof(str), fp1) != NULL)
39     {
40         /* Clear token arrays before using them */
41         s1[0] = s2[0] = s3[0] = '\0';
42     }
```



Output:


```
✓ TERMINAL
PS C:\Users\Shruti> cd "c:\Users\Shruti\OneDrive\Desktop\Spcc\" ; if ($?) { g++ ASSPASS1.C -o ASSPASS1 } ; if ($?) { .\ASSPASS1 }

Symbol Table
PG1 1000
FOUR 1016
TEMP 1020

Literal Table
='5' 1024
='4' 1028
PS C:\Users\Shruti\OneDrive\Desktop\Spcc>
```

B.3 Observations and learning:

Pass 1 of an assembler is a foundational phase that establishes the groundwork for subsequent stages of the assembly process. Effective symbol management, error detection, and collaboration with other phases contribute to the successful generation of the intermediate representation required for creating the final machine code. Understanding the intricacies of Pass 1 is essential for efficient and accurate assembly of programs.

B.4 Conclusion:

In conclusion, Pass 1 of an assembler is a critical phase in the assembly process that lays the groundwork for the subsequent stages. The primary tasks performed during this phase include the creation of a symbol table, updating the location counter, processing assembler directives, handling literals, and performing early error detection.

B.5 Question of Curiosity

A. Define Data Structures

1. Mnemonic Operation Table

Ans The term "Mnemonic Operation Table" typically refers to a data structure used in computer architecture or assembly language programming. It is designed to map mnemonic instructions to their corresponding machine code or binary representations. In assembly language programming, mnemonics are human-readable representations of instructions that are easier for programmers to understand than machine code. The Mnemonic Operation Table serves as a lookup reference for programmers and assemblers. It facilitates the translation from human readable mnemonics to the machine-understandable OpCode, aiding in the

programming process. Additionally, the table may include information about the number of operands, addressing modes, and other details relevant to the instruction set architecture. Here is a hypothetical example of a Mnemonic Operation Table:

Mnemonic	OpCode	Operand(s)	Format
ADD	0001	Reg, Reg	R-Type
SUB	0010	Reg, Imm	I-Type
LOAD	0100	Mem Addr	Load/Store
STORE	0101	Mem Addr	Load/Store
JMP	1000	Jump Addr	Jump

In this example, the Mnemonic Operation Table provides a mapping between mnemonic instructions (such as ADD, SUB, LOAD, STORE, and JMP) and their corresponding OpCode values, operands, and instruction formats. This facilitates the assembly and programming process by offering a clear reference for the translation of high-level instructions to machine code.

2. Symbol Table

Ans A Symbol Table is a data structure used in computer science and programming to manage and store information about symbols, such as variables, functions, labels, or constants, within a program. The primary purpose of a Symbol Table is to associate each symbol with relevant information, such as its memory address, data type, scope, and other attributes. Symbol Tables play a crucial role in the compilation and execution of programs, aiding in the management of identifiers and their corresponding properties.

3. Segment Register Table

Ans A segment register changes the memory address accessed by 16 bits at a time, because its value is shifted left by 4 bits (or multiplied by 16) to cover the entire 20-bit address space. The segment register value is added to the addressing register's 16-bit value to produce the actual 20-bit memory address.

4. Forward Reference Table

Ans A forward reference occurs when a label is used as an operand, for example as a branch target, earlier in the code than the definition of the label. The assembler cannot know the address of the forward reference label until it reads the definition of the label.

5. Cross Reference Table

Ans A Cross Reference Table is a database Table that links rows in two or more tables. This kind of database table is a common concept in databases in general. This page discusses various netFORUM nuances with this kind of table.

B. Comment on the Forward Reference Problem and Remedy.

Ans The forward reference problem is a common challenge in programming and compilation, and it arises when a symbol (such as a variable or function) is used before it is declared or defined in the code. This situation can lead to difficulties in the compilation

process because the compiler needs to know the characteristics and location of symbols to generate correct machine code. If the compiler encounters a reference to a symbol before it has been declared or defined, it faces the forward reference problem.

Forward Reference Problem:

1. Compilation Dependency:

- The compiler relies on a sequential process of parsing the source code. If a symbol is referenced before its declaration or definition, the compiler may not have sufficient information to generate the correct machine code.

2. Incomplete Symbol Information:

- Without prior knowledge of a symbol's data type, size, or location, the compiler cannot perform certain optimizations and generate code that adheres to the correct memory layout.

3. Linking Issues:

- Forward references can also affect the linking phase if not properly resolved. Linkers need complete information about symbols to connect different object files or modules into a coherent executable.

Remedy for the Forward Reference Problem:

1. Declaration Statements:

- One common remedy is to use declaration statements to inform the compiler about the existence of symbols before they are used. For example, function prototypes can be declared at the beginning of a file or in header files.

```
// Function prototype
void myFunction(int arg);

// Function definition later in the code
void myFunction(int arg) {
// Function implementation
}
```

2. Header Files and Modules:

- In modular programming, header files or module interfaces can declare the structure of functions or variables before their actual definitions. This allows different modules to interact without forward reference issues.

```
// Header file (example.h)
void myFunction(int arg);

// Source file (example.c)
#include "example.h"

void myFunction(int arg) {
```

```
// Function implementation  
}
```

3. Use of External Linkage:

- When symbols are defined in different files, the use of external linkage (using `extern` in C, for example) can help the compiler understand that the actual definition exists elsewhere.

```
// File1.c  
int globalVariable;
```

```
// File2.c  
extern int globalVariable; // Declaration, providing forward reference
```

4. Forward Declarations:

- Forward declarations are often used for functions or classes. They provide the compiler with enough information to generate code even if the full definition comes later in the code.

```
// Forward declaration  
class MyClass;
```

```
// Usage  
MyClass* obj = new MyClass();
```

5. Order of Code:

- Organizing the code in a way that definitions precede their usage within the same file or module can also address forward reference problems.